

FabJudge: JEV-LLM은 이상 탐지에서 Edge를 만들 수 있는가?

PHM2018 기반 RF-LSTM Baseline과 선택적 판단 계층의 검증

작성: 이윤민 · 2026-09-27 · 이 보고서의 모든 결과는 저장된 실험 artifact에 한정된다. API 비용은 해당 실행 시점의 기록값이다.

GitHub · github.com/LeeYmin/FabJudge-PHM2018 — 코드, 실행 노트북, 결과 표와 그림

0. 요약

이 프로젝트는 단순한 질문에서 출발했다. 학습된 RF-LSTM 이상 탐지 파이프라인 위에 JEV(수치 판단 특화 모델)와 LLM(GPT-6 Luna)을 판단 계층으로 올리면 탐지나 경보 우선순위가 좋아질까? 비교 기준으로는 Huang et al. (2018)의 2단 구조를 참고해 만든 RF-LSTM baseline(04b)을 썼다. 04b는 원 논문을 그대로 복제한 것이 아니다.

결론부터 말하면 판단 계층의 우위는 확인하지 못했다. JEV 단독 실험(05~06 시리즈)에서 입력 표현, 스케일, 프롬프트 문구를 바꾸고 가설 프롬프트 10종을 고정된 검증기로 시험했지만 모두 기준을 넘지 못했다. 사전 등록된 확증 실험(07b)에서도 첫 가설(H1)부터 기각에 실패했다. 탐색 분석에서 JEV·LLM은 같은 입력으로 학습한 로지스틱 회귀보다 낮았다.

대신 판단 계층을 검증하는 과정에서 더 실용적인 사실을 하나 찾았다. **04b의 RF 게이트가 LSTM이 이미 잡은 실제 고장 정보를 버리고 있었다.** fault3에서 endpoint recall은 04b 0.629, 게이트를 빼면 0.974였고, fault1+2에서는 평가 가능한 고장 사건 52건 중 6건을 04b만 놓쳤다.

그래서 이 프로젝트가 최종적으로 지지하는 설계 원칙은 **경보 보존과 판단 지원의 분리**다. LSTM 경보는 없애지 않고, RF는 검토 tier를 나누는 신호로 쓰며, JEV와 LLM은 검토 순서와 설명의 깊이만 돕는다.

LLM(GPT-6 Luna)은 판별에서 이기지 못했지만 운영 비용은 낮았다. 경보당 평균 약 \$0.00026, 재시도 후 유효 응답 99.6%, 인용한 수치 1,706개가 모두 원 입력과 일치했다. 경보를 취소할 권한이 없는 저비용 참고 계층으로는 둘 수 있다. 다만 사람의 판단에 실제로 도움이 되는지는 측정하지 않았다.

1. 배경과 질문

PHM Society 2018 Data Challenge는 이온 밀링 식각 장비의 센서 시계열로 세 가지 고장 모드(fault1/2/3)의 발생 시점을 예측하는 문제다. 공식 문제 정의는 예측 시점의 현재와 과거 데이터만 쓰도록 요구하고, 공개 데이터의 센서 물리 단위는 익명화되어 있다. 이 보고서에서는 실제 고장까지 남은 시간(TTF)이 5,000초 이하인 시점을 **임박 고장**으로 보고, 이를 얼마나 놓치지 않고 경보하는지를 평가했다.

비교 기준인 04b는 RF 확률과 LSTM의 잔여시간 예측을 결합한다. 경보는 **LSTM 예측 $\leq 5,000$ 초이고 RF 확률 ≥ 0.5 일 때**만 울린다. 여기서 RF는 5k 이내 fault와 50k 초과 normal을 구분하도록 학습된 분류기이며, LSTM 경보를 통과시킬지 결정하는 게이트 역할을 한다.

Huang et al. (2018)은 급작스러운 고장이 있는 시스템에서 먼저 열화 모드를 탐지하고, 그 뒤 LSTM으로 RUL을 추정하는 2단 구조를 제안했다. 04b는 이 구조와 5k/50k 구분 아이디어를 출발점으로 삼았다. 하지만 저장된 기록만으로는 원 논문의 전처리, 분할, 하이퍼파라미터, 공식 평가까지 똑같이 재현했다고 말할 수 없어서, 이 보고서에서는 **Huang-inspired RF-LSTM baseline**이라고 부른다. 뒤에 나오는 "04b 대비" 수치는 모두 이 프로젝트 안에서 고정된 04b 결과를 기준으로 한 것이다. 원 논문이나 공식 challenge 성능을 넘었다는 뜻이 아니며, 원 논문의 주 목표는 RUL 정확도였던 반면 이 보고서의 비교 축은 경보 보존과 우선순위다.

질문은 두 가지였다.

- **Q1.** JEV-LLM 판단 계층이 04b보다 이상 탐지나 경보 우선순위에서 나은가?
- **Q2.** 낫지 않다면, 기존 학습 모델을 대체하지 않으면서 어떤 역할을 맡길 수 있는가?

JEV는 OpenRouter로 제공되는 외부 판단 API로, 동료 심사를 거친 방법론으로 다루지 않는다. 논문에서 가져온 부분, 공개 코드 참고, 공식 데이터·문서, 이 프로젝트에서 새로 설계한 부분은 9장에서 따로 구분했다.

근거: notebooks/04b_huang2018_reproduction.ipynb · src/fabjudge/huang2018_models.py · artifacts/huang2018/split_metadata.json · artifacts/08_final_comparison/tables/T5b_04b_original_metrics.csv

2. 시스템과 평가 설계

2.1 구조

그림 S1은 이 보고서에서 비교하는 두 경로다. 위쪽(A)이 04b, 아래쪽(B)이 경보를 버리지 않는 구조다. 3장의 수치는 결국 A가 버린 경보가 B의 rescue lane에 무엇을 남기는지에 관한 이야기다.

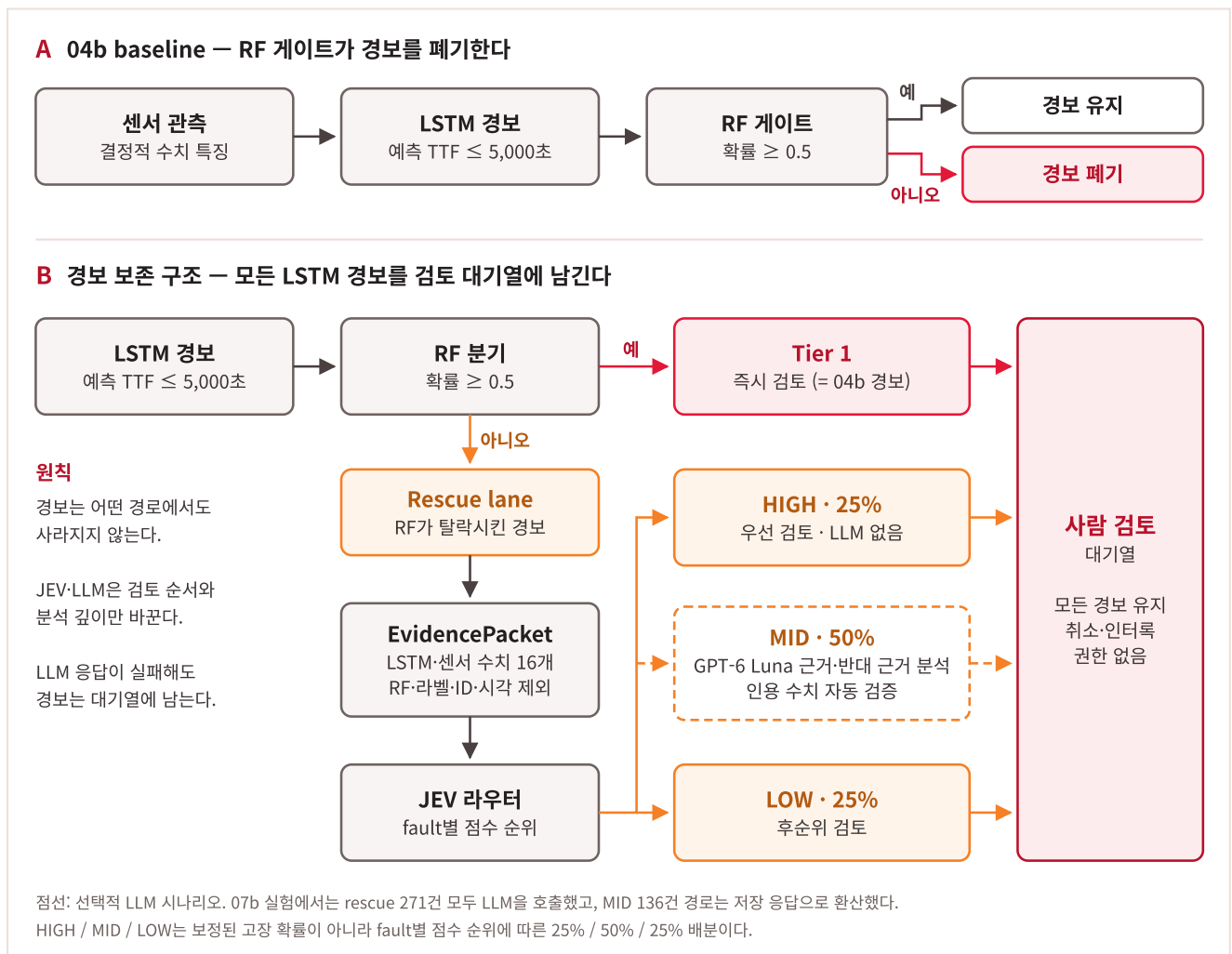


그림 S1. 04b와 경보 보존 구조. (A) 04b는 RF 확률이 0.5 미만이면 기존 LSTM 경보를 폐기한다. (B) 제안 구조는 04b 경보를 tier1으로 바로 검토하고, RF가 탈락시킨 LSTM 경보는 rescue lane에 남긴다. rescue 경보는 JEV 점수로 fault별 HIGH 25% / MID 50% / LOW 25%로 나뉘고, LLM(GPT-6 Luna)은 근거 요약과 우선순위를 만든다. 모든 경로는 사람 검토 대기열로 끝나며 판단 계층에는 경보 취소나 인터록 권한이 없다. 점선은 선택적 LLM 시나리오다. 07b 실험에서는 rescue 271건 전부에 LLM을 호출했고, MID 136건만 호출하는 경로는 저장된 응답으로 환산했다.

원본 그림: docs/figures/S1_alarm_preserving_pipeline.png (scripts/draw_paper_pipeline.py). 이 보고서에는 같은 구조를 가독성을 위해 다시 그려 넣었다. · 실행 규칙: artifacts/07b_rescue_lane_confirmatory/prereg_07_v2_1.json

2.2 입력과 출력, 그리고 누수 방지

JEV와 LLM(GPT-6 Luna)은 서로 다른 API 키와 로더를 쓰고, 한쪽 키가 없을 때 다른 키로 넘어가는 경로를 두지 않았다. 판단 계층에 들어가는 입력(EvidencePacket)에는 정답 TTF, 라벨, 파일·시퀀스 식별자, 시각, 표본 위치를 넣지 않았다. 최종 07b 입력은 RF 확률을 뺀 16개 필드다. RF가 이미 "아니다"라고 한 경보에 RF 확률을 다시 주면 모델이 그 판단을 그대로 따라갈 가능성이 크다고 봤기 때문이다. LLM 출력은 근거, 반대 근거, 미확인 사항, 추가 확인, 우선순위(0~100), 검토 권고로 제한했고, 인용한 필드와 값은 코드로 원 입력과 대조했다.

2.3 평가 원칙

먼저 분석 단위를 분명히 해 둔다. 평가 단위는 센서 원시 행이 아니라 04b가 예측을 낸 시점인 **endpoint**다. 같은 고장 시퀀스에서 나온 endpoint 여러 개는 독립적인 고장 사건이 아니므로, endpoint recall(임박 고장 시점을 얼마나 회수했나)과 사건 탐지(평가 가능한 고장 시퀀스 중 한 번이라도 경보했나)를 따로 보고한다. 이 둘을 섞으면 수백 개의 endpoint 회수를 수백 건의 고장 예방처럼 잘못 읽게 된다.

평가는 두 층으로 나뉜다. 첫째는 **확증 실험(07b)**이다. 가설과 규칙을 미리 등록하고 파일 해시를 고정한 뒤, 라벨을 열기 전에 순위를 확정했다. 가설은 H1 → H2 → H3 순서로 검정하고, 앞의 가설이 통과해야 다음으로 넘어간다($\alpha = 0.05$, 시퀀스 단위 bootstrap 10,000회). 표본은 test 시퀀스마다 LSTM 경보 구간의 고정 percentile 10개 지점을 뽑은 720행이며, 주 모집단은 fault1+2 rescue 249행(양성 32, 음성 217, 시퀀스 37)이다. fault3는 따로 보고했다.

둘째는 **전체 endpoint 비교(08)**다. 저장된 04b test 예측 전체(fault1 13,827, fault2 5,053, fault3 2,416 endpoint)를 API 호출 없이 다시 집계한 사후 기술 분석이다. 여기서 "전체"는 원시 센서의 모든 행이 아니라 저장된 causal test endpoint 전부를 뜻한다.

근거: artifacts/07b_rescue_lane_confirmatory/{prereg_07_v2_1.json, trial_log.md} · artifacts/08_final_comparison/tables/{07b_integrity_checks.csv, full_endpoint_coverage.csv, final_consistency_checks.csv}

3. 핵심 발견: RF 게이트가 실제 고장 경보를 버린다

이 장의 비교에는 저장된 04b 예측과 고정 임계값(RF 0.5, LSTM 5,000초)만 쓰였다. test에 맞춰 다시 학습하거나 임계값을 바꾼 것은 없다.

3.1 fault3: 게이트가 치르는 비용

표 1. fault3 전체 endpoint 비교

방식	양성 endpoint	TP	FN	검토 endpoint	Recall	Precision
04b (RF 게이트)	466	293	173	439	0.629	0.667
LSTM 경보 전부 유지 (tier1 + rescue)	466	454	12	747	0.974	0.608

fault3는 LSTM이 전체 2,416개 endpoint 중 747개에서만 경보를 내는, 즉 LSTM이 나름대로 선별을 하는 fault다. 그런데 RF가 탈락시킨 308개 중 161개(52.3%)가 실제로 5,000초 안에 고장 난 시점이었다. 추가로 약 1.9개를 검토할 때마다 양성 endpoint 하나를 되찾는 셈이다(독립적인 고장 사건 하나와 같지는 않다). 정리하면 RF 게이트는 recall 34.5%p를 내주고 precision 6%p를 얻는 거래를 하고 있었다.

사건 단위로 봐도 비슷한 모습이 나온다. fault3 8개 시퀀스에서는 두 방식 모두 6개를 잡았지만, 첫 경보 시점의 중앙값이 04b는 고장 전 3,815초, 게이트 없이는 4,949초였다. 같은 사건도 게이트를 거치면 약 1,100초 늦게 알게 된다.

3.2 fault1+2: 사건 자체를 놓친다

표 2. 시퀀스(사건) 단위 탐지

범위	전체 시퀀스	평가 가능	04b 탐지	LSTM 전부 유지 탐지	04b만 놓친 사건
fault1	48	39	35	39	4
fault2	18	13	11	13	2
fault1+2	66	52	46	52	6
fault3	8	6	6	6	0

평가 가능 = 저장 endpoint 중 TTF ≤ 5,000초 시점이 하나 이상 있는 시퀀스. 나머지 14개(fault1 9, fault2 5)는 이 정의로 평가할 수 없어 분 모에서 뺐다. 이들을 정상 사건이나 탐지 성공으로 취급하지 않았고, 원인을 일괄적으로 추정하지도 않았다.

3.3 fault1/2에서는 판별 자체가 약하다

표 3. fault1+2 전체 endpoint 비교 (임박 고장 기저율 0.165)

방식	검토 endpoint	04b 대비 추가	Recall [95% CI]	Recall 변화 [95% CI]	Precision
04b	11,754	0	0.704 [0.587, 0.817]	—	0.187
04b + rescue 무작위 E10	12,466	712	0.733	+3.0%p [1.7, 4.2]	0.183
04b + rescue 무작위 E20	13,180	1,426	0.763 [0.669, 0.853]	+5.9%p [3.5, 8.4]	0.181
04b + rescue 무작위 E30	13,892	2,138	0.793	+8.9%p [5.3, 12.6]	0.178
LSTM 경보 전부 유지	18,880	7,126	1.000	+29.6%p	0.165

표 3을 읽을 때는 한 가지를 먼저 알아야 한다. fault1+2에서 LSTM은 저장된 test endpoint 18,880개 **전부**에 경보를 낸다. 그래서 마지막 행의 recall 1.000은 사실상 "전부 검토하면 전부 잡는다"는 뜻이다. 04b의 precision도 0.165보다 2%p 남짓 높을 뿐이다. 즉 이 두 fault에서는 경보 판별력 자체가 약하고, RF가 버린 경보 속에도 검토할 만한 고장이 여전히 남아 있다. 무작위 E 행은 rescue를 무작위 순서로 검토할 때의 기대값이다. 4장에서 보듯 JEV·LLM 정렬이 무작위와 구분되지 않았기 때문에 이 값을 파이프라인 성능의 추정치로 썼다. 실제 판단 계층의 호출 결과는 아니다.

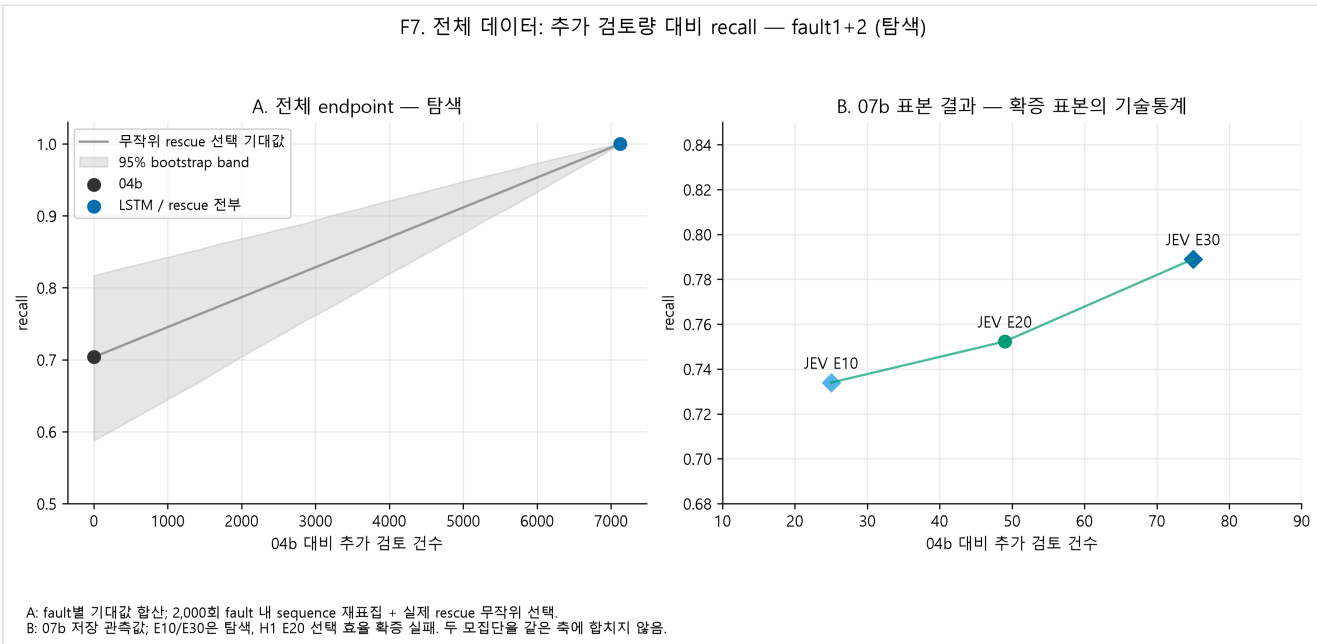


그림 F7. 추가 검토량 대비 recall (fault1+2). (A) 전체 endpoint에서 rescue를 무작위로 검토할 때의 기대 recall과 95% bootstrap 구간. (B) 07b 확증 표본에서 JEV 라우팅의 실제 결과. 모집단이 달라 두 패널을 한 축에 합치지 않았다.

3.4 해석

이 파이프라인에서는 RF를 경보를 버리는 조건이 아니라 tier와 검토 순서를 정하는 신호로 쓰는 편이 낫다고 판단한다. 그림 S1의 B 구조가 바로 그 변경이고, 3.1~3.3은 그렇게 했을 때 되살아나는 경보의 규모다. 필요한 검토 인력과 실제 운영 효과는 별도로 확인해야 한다.

근거: artifacts/08_final_comparison/tables/{T5_full_endpoint_comparison.csv, T6_sequence_event_detection.csv, T6_sequence_event_details.csv, F7_full_endpoint_curve.csv} · artifacts/huang2018/fault{1,2,3}_test_predictions.parquet · notebooks/08_final_comparison.ipynb

4. 판단 계층의 판별력: 학습된 소형 모델을 넘지 못했다

3장의 발견은 사실 판단 계층을 시험하다가 얻은 부산물이다. 이 장은 원래 질문, 즉 JEV와 LLM이 판별이나 우선순위에서 이길 수 있는지를 시간 순서대로 따라간다. 05 시리즈에서는 JEV 하나만 놓고 입력과 프롬프트를 다듬었고, 06 시리즈에서는 JEV를 LSTM 경보 뒤에 붙였으며, 07b에서 LLM까지 포함해 확증 검정을 했다.

4.1 JEV 단독 탐색 (05 시리즈): 입력 표현, 스케일, 프롬프트

입력 표현(05a). 처음에는 JEV가 센서 요약값을 어떤 형태로 받아야 이상 상태를 알아보는지부터 확인했다. validation 그룹에서 세 개 target(FlowCool 압력 low, 압력 high, leak) × 세 구간(정상 TTF > 50k, 경계, 임박 ≤ 5k) × 5개씩 모두 45개 창을 뽑고, 같은 20개 feature를 원값(raw), z-score, train 기준 Robust 변환 세 가지로 보냈다. 질문(v4)은 "이 장비 상태가 얼마나 비정상적이고 고장에 가까운가"를 0~2 점수로 묻는 일반적인 문구였다.

표 4. 05a 입력 표현별 임박 대 정상 ROC-AUC (target별, n = 15)

입력 표현	압력 low	압력 high	leak
raw	0.56	0.82	0.24
z-score	0.64	0.36	1.00
Robust	0.88	0.86	1.00

원값을 그대로 주면 JEV는 거의 구분하지 못했고, Robust 변환을 했을 때만 세 target 모두에서 정상 < 경계 < 임박 순서가 나왔다. 이후 모든 JEV 실험은 train 기준 Robust 변환을 기본으로 삼았다. 다만 target당 15개라는 작은 표본이었다는 점은 뒤에서 다시 문제가 된다.

스케일 민감도. JEV가 숫자의 크기에 얼마나 민감한지도 따로 시험했다. 같은 45개 창에서 FLOWCOOLPRESSURE__shape_factor 하나만 0.1배, 0.5배, 2배, 10배로 바꾸고 나머지 19개는 그대로 두었다.

표 5. 단일 feature 스케일 변경에 따른 macro ROC-AUC

배율 (배치)	임박 - 정상 점수 차	ROC-AUC	순서가 맞은 target
0.1× (신규)	0.249	0.913	3/3
0.5× (기존)	0.242	0.927	2/3
1× (기존)	0.249	0.973	3/3
1× (신규 대조군)	0.255	0.933	3/3
2× (기존)	0.230	0.933	2/3
10× (신규)	0.216	0.880	1/3

어떤 배율도 1배보다 낮지 않았다. 0.1배는 신규 1배와 거의 같았고(점수 차 변화 -0.006, 95% 구간 -0.024~0.011), 10배는 분명히 나빠졌다(-0.039, -0.067~-0.011). 흥미로운 점은 같은 1배 조건을 다시 돌렸을 때도 점수가 평균 0.021 움직였다는 것이다. 작은 배율 차이는 이 반복 변동 안에 묻힌다. 결론적으로 스케일 조작은 개선 수단이 아니었고, 1배 Robust 입력을 유지했다.

프롬프트 수정과 확대 표본(05b). 다음으로 질문 문구를 고쳤다. v5는 "하나의 값만 보고 판단하지 말고 전체 패턴과 근거의 강도를 보라, 근거가 섞여 있으면 Uncertain을 고르라"는 지시를 더한 버전이다. 이번에는 05a에서 쓴 그룹을 빼고 270개 창

(개발 180, holdout 90, 그룹 중복 0)으로 규모를 키웠고, "v5는 개발 세트의 네 조건을 모두 통과할 때만 채택한다"는 규칙을 먼저 잡았다.

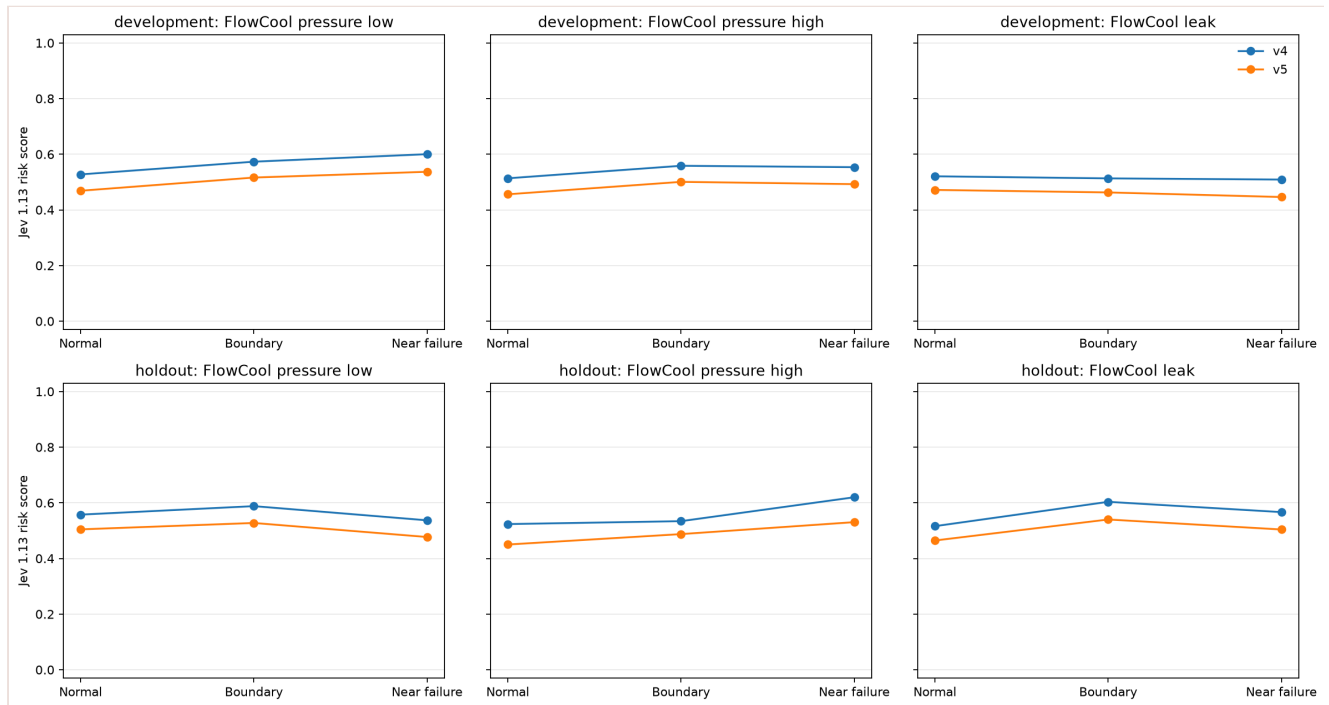


그림 J1. 05b 개발·holdout 세트의 구간별 평균 JEV 점수 (v4 대 v5). 270개 창으로 표본을 키우자 정상, 경계, 임박 구간의 점수 차이가 거의 사라졌다. v5는 v4보다 전반적으로 점수를 낮출 뿐 구분을 개선하지 못했다.

결과는 실망스러웠다. v5는 개발 세트에서 구분 폭이 오히려 줄어(0.034 → 0.026) 사전 규칙상 탈락했다. 더 큰 문제는 기준인 v4 자체였다. 05a에서 0.234였던 임박 - 정상 점수 차가 270개 창에서는 0.037로 줄었고, macro ROC-AUC는 개발 0.602, holdout 0.622에 그쳤다. 두 코호트는 창이 겹치지 않는 비교라 원인을 단정할 수는 없지만, 적어도 05a의 인상적인 숫자가 작은 표본에서 나온 낙관이었을 가능성을 보여 준다. 이 점수로 세 구간 임계값을 잡아 봤지만 그룹 교차검증 balanced accuracy는 0.372로, 무조건 한 등급만 찍는 기준(0.333)과 큰 차이가 없었다.

가설 프롬프트 10종(05c). 문구를 조금씩 고치는 방식으로 한계가 있다고 보고, 마지막으로 "JEV가 왜 틀리는가"에 대한 가설을 하나씩 프롬프트로 만들어 시험했다. 가설마다 v4가 틀린 사례를 근거로 삼았고, 결과를 보기 전에 검증기(v1.2)와 통과 기준을 해시로 고정했다. 후보는 v4와 같은 요청 안에서 짝지어 비교했고(45개 창, 40개 그룹, 3-fold 그룹 교차검증), 다음을 모두 만족해야 살아남도록 했다.

- ROC-AUC가 v4보다 0.02 이상 높고 0.62 이상이거나, macro-F1이 0.03 이상 높고 0.45 이상일 것
- 동시에 ROC-AUC와 macro-F1이 v4보다 0.01 넘게 떨어지지 않을 것
- 임박 고장을 정상으로 보는 오류가 늘지 않고, 정상을 임박으로 보는 비율이 5%p 넘게 늘지 않을 것

표 6. 05c 가설 프롬프트 테스트케이스와 판정

#	가설 (프롬프트에 추가한 해석 규칙)	ROC-AUC 후보 / v4	macro-F1 후보 / v4	판정
1	feature 중심에서 위아래 어느 쪽으로 벗어나도 이상으로 본다	0.567 / 0.613	0.316 / 0.327	탈락
2	같은 센서에서 나온 요약값은 하나의 근거로 센다	0.607 / 0.627	0.370 / 0.367	탈락
3	순간 peak보다 지속되는 평균·RMS를 우선한다	0.607 / 0.607	0.360 / 0.370	탈락
4	수준 변화와 창 내부 변동성을 별개 신호로 본다	0.620 / 0.600	0.355 / 0.405	탈락
5	shape_factor는 장비 수준이 아니라 분포 모양으로 해석한다	0.667 / 0.640	0.337 / 0.485	탈락
6	skewness를 꼬리 비대칭의 단서로 쓴다	0.567 / 0.600	0.355 / 0.434	탈락
7	냉각 압력과 유량을 결합된 관계로 해석한다	0.613 / 0.580	0.405 / 0.551	탈락
8	점수 0/1/2를 명시적 수치 기준에 묶는다	0.573 / 0.620	0.395 / 0.325	탈락
9	센서별 물리적 의미로 높음/낮음의 방향을 판단한다	0.593 / 0.613	0.435 / 0.447	탈락
10	등급 안에서도 세밀한 점수 차이를 유지한다	0.620 / 0.633	0.446 / 0.332	탈락

v4 값이 시험마다 다른 것은 v4를 각 후보와 같은 요청에 넣어 매번 새로 받았기 때문이다. 10회 합계 450회 호출, 기록 비용 \$0.021.

열 개 모두 탈락했다. 몇몇 가설(4, 5, 7)은 ROC-AUC를 조금 올렸지만 그 대가로 macro-F1이 크게 떨어지거나 위험한 오류가 늘었다. 반대로 8, 10번은 F1은 올렸지만 ROC-AUC를 잃었다. 한쪽을 고치면 다른 쪽이 무너지는 패턴이 반복됐다. 마지막 확인 단계는 아예 진행할 수 없었는데, 05b까지 쓰고 남은 임박 고장 그룹이 하나도 없었기 때문이다(판정 INCONCLUSIVE). 같은 개발 창을 열 번 재사용했으니 선택 편향도 있다.

05 시리즈에서 얻은 교훈은 세 가지다. 입력 표현은 중요했지만(Robust 변환), 그 이상의 조정, 즉 스케일이나 문구, 해석 규칙은 효과가 없었다. 작은 표본의 좋은 결과는 표본을 키우자 대부분 사라졌다. 그리고 JEV가 수치 패턴에서 뽑아낼 수 있는 정보에는 이 데이터에서 분명한 상한이 있어 보였다. 이 판단을 안고 JEV를 LSTM 경보 뒤에 붙이는 06 시리즈로 넘어갔다.

근거: artifacts/jev_tut/{config.json, representation_summary.csv, prompt_v4.txt} · artifacts/jev_scale_sensitivity/{macro_comparison.csv, extreme_0p1_10x/summary.md} · artifacts/jev_prompt_optimization/{run_summary.md, decision_questions.json, macro_comparison.csv, development_selection.json, diagnostics/summary.md} · artifacts/jev_v4_edge_trials/{trial_log.md, trial_summary.csv, hypothesis_registry.csv, validation_spec.json, final_confirmation.json}

4.2 LSTM 경보 뒤의 JEV (06 시리즈)

표 7. 06 시리즈 경과

단계	질문	핵심 수치	결론
06a·06b	JEV가 LSTM 경보의 FP를 걸러내는가	제거 경보 0건	보수적 문구에서 점수가 모두 임계값 이상
06c	dev로 정한 임계값이 test에서 유지되는가	test recall: P2 0.500, LSTM 0.938	in-sample 성격의 dev에서 잡은 임계값이 무너짐
06d	OOF RF와 결합하면 독립 신호가 있는가	J2 dev AUC 0.831 무효	dev 요청에 in-sample RF가 들어간 버그
06e	버그 정정과 약점 진단	정정 후 J2 dev AUC 0.593	강한 독립 신호 없음. 긴 창 feature는 검증 불가

06 시리즈는 한동안 희망적으로 보였다. 06d에서 JEV와 RF를 결합하자 dev AUC가 0.83까지 올라갔기 때문이다. 하지만 원인을 추적해 보니 dev 요청에 교차적합이 아닌 in-sample RF 확률이 들어가 있었다. 버그를 고치자 같은 값은 0.59로 떨어졌다. 이 경험 이후 모든 실험에서 입력 필드를 코드로 검증하는 절차를 넣었다.

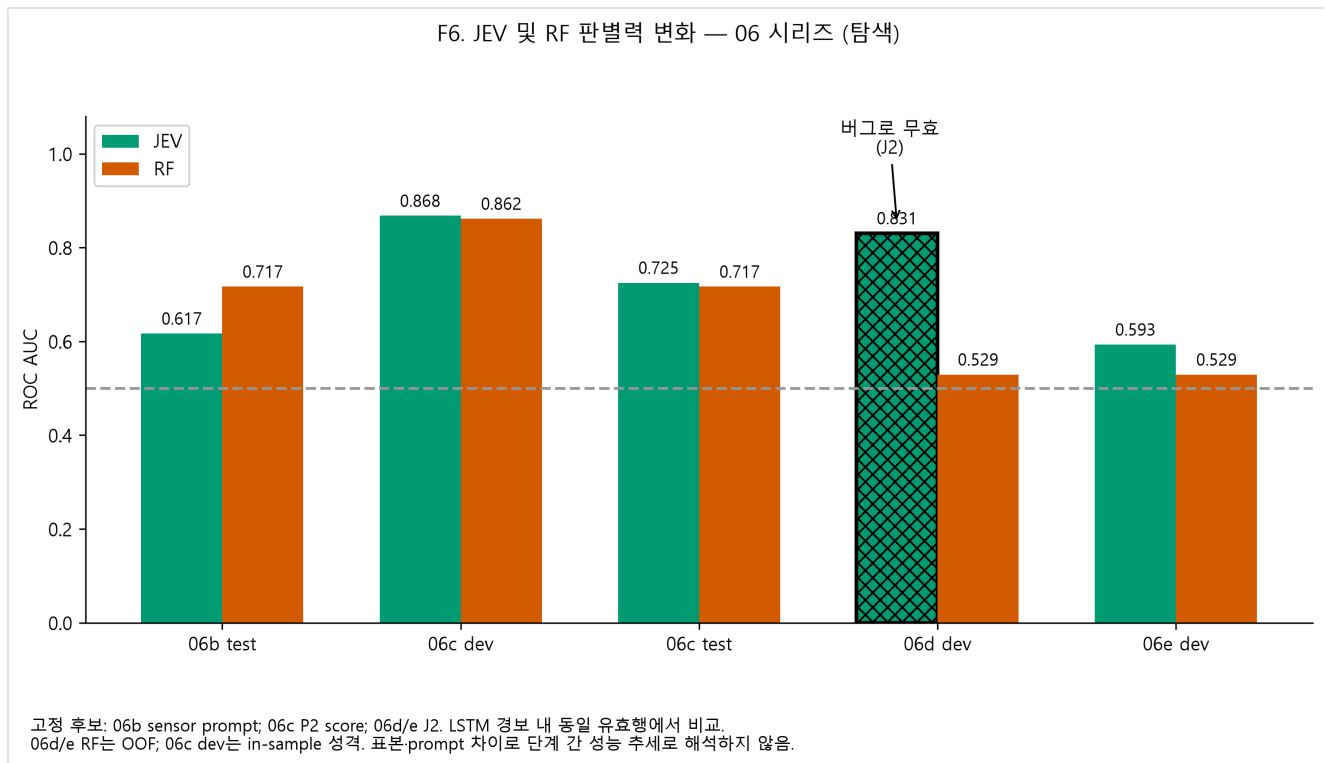


그림 F6. 06 시리즈의 JEV와 RF AUC. 06d의 JEV 값은 누수 버그로 무효다. 단계마다 표본과 모집단이 달라 같은 모집단의 성능 추세로 읽으면 안 된다.

근거: artifacts/08_final_comparison/tables/{T3_experiment_history.csv, F6_auc_source_data.csv} · artifacts/06d_lstm_jev_oof_edge/ERRATUM.md · artifacts/06e_fix_and_weakness_diagnosis/{dev_metric_comparison.csv, conclusion.md}

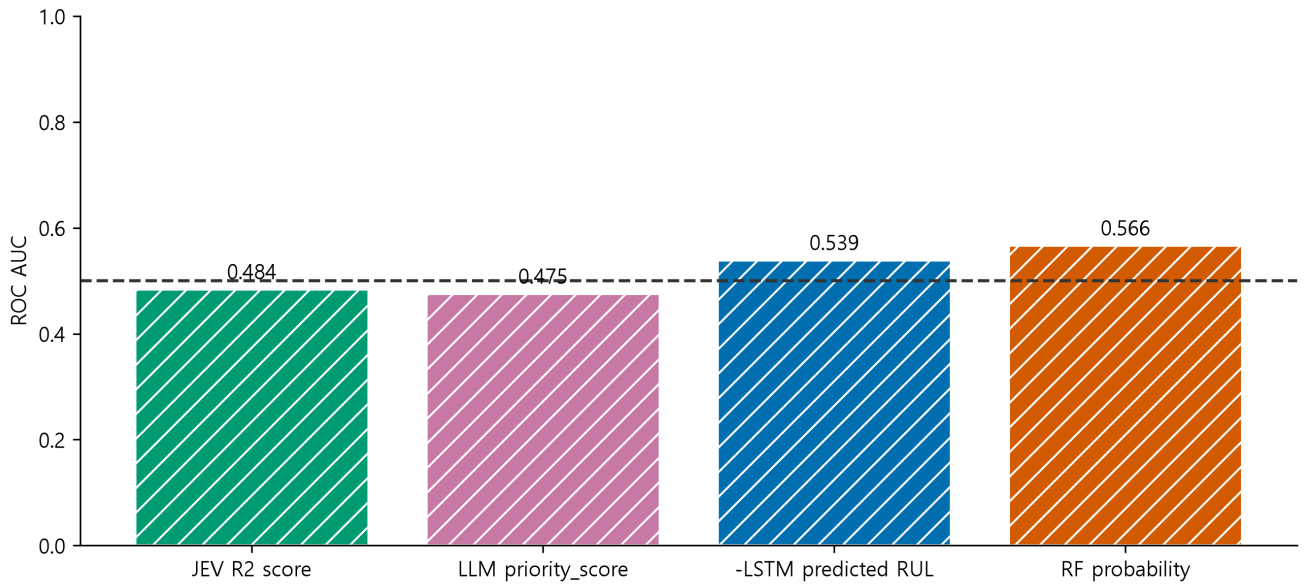
4.3 확증 검정 (07b, LLM: GPT-6 Luna)

표 8. 사전 등록 가설 검정 (fault1+2 rescue, E20) 확증

가설	추정치	비교값	차이	단측 95% 하한	판정
H1: JEV 정렬 precision@E20 > rescue 기저율	0.102	0.129	-0.026	-0.072	기각 실패
H2: JEV 라우팅 - 전체 LLM > -0.05 (비열등)	0.102	0.102	0.000	—	미검정
H3: JEV 라우팅 > 무작위 라우팅	0.102	0.130	-0.028	—	미검정

마지막 확증 실험에서 JEV 정렬은 rescue 기저율조차 넘지 못했다. E20 예산으로 검토한 49행 중 실제 임박 고장은 5행이었다(precision 0.102). 같은 표본 안에서 04b 대비 recall은 0.706에서 0.752로 올랐지만, 이는 검토량을 늘린 효과이지 정렬이 좋아서가 아니다. H1이 통과하지 못했으므로 순차 규칙에 따라 H2, H3는 검정하지 않았다. 기각 실패는 "개선의 근거를 찾지 못했다"는 뜻이지 "무작위와 같다"를 증명한 것은 아니다.

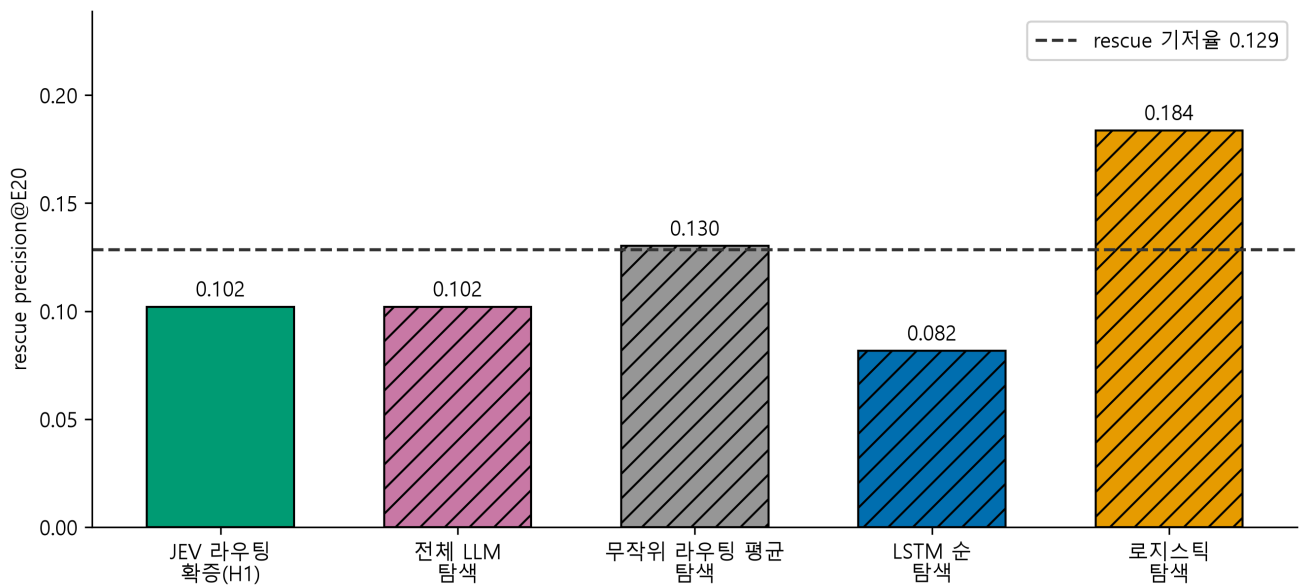
F4. rescue 내부 각 신호의 구분력 — 탐색



fault1+2 rescue; endpoint key로 일대일 join. LLM MODEL_UNAVAILABLE 1건 제외.
LLM n=248, 다른 신호 n=249; 표본 차이 주의.

그림 F4. rescue 안에서 각 신호의 구분력 (fault1+2) 탐색 AUC는 JEV 0.484, LLM(GPT-6 Luna) 0.475, LSTM 0.539, RF 0.566이다. LLM은 응답 실패 1건을 뺀 248행, 나머지는 249행 기준.

F2. rescue precision@E20 비교 — 확증 H1 및 탐색 분석



fault1+2; H1 실패. H2/H3는 gatekeeping 미검정. 빗금 막대는 탐색 비교.
LSTM 순 = LSTM_only (LLM 미사용); 무작위 라우팅 = 저장된 1,000회 평균.

그림 F2. rescue precision@E20 비교 (fault1+2). JEV 라우팅만 확증(H1) 대상이고 나머지는 탐색이다. 로지스틱 0.184, 전체 LLM 0.102, LSTM 예측값 순 0.082. LLM이 모든 지표에서 LSTM보다 낮았던 것은 아니다.

4.4 해석

같은 16개 수치를 받은 로지스틱 회귀와 비교해도, 이 표본에서 범용 모델이 앞선다는 증거는 없었다. 05 시리즈부터 이어진 흐름을 보면 JEV와 LLM이 이 데이터에서 끌어낼 수 있는 신호에는 한계가 있었고, 그 신호는 해당 데이터로 직접 학습한 작은 모델이 더 잘 잡았다. 다만 RF-LSTM과의 AUC 비교는 입력을 완전히 통제된 실험이 아니므로, "모델 크기가 아니라 입력 정보가 원인"이라고 단정하지는 않는다. 그 질문에 답하려면 입력과 모델을 따로 통제된 실험이 더 필요하다.

근거: artifacts/07b_rescue_lane_confirmatory/{hypothesis_tests.json, exploratory.csv, rankings.csv} · artifacts/08_final_comparison/tables/{T2_confirmatory_hypothesis_tests.csv, F4_auc_source_data.csv}

5. 낮은 비용의 참고 계층으로서의 역할

판단 계층이 서 있는 자리는 그림 S1의 rescue 경로 안쪽이다. 이미 살아남은 경보에 순서와 설명을 붙이는 단계일 뿐, 경보를 남길지 말지는 결정하지 않는다. 판별에서 이기지 못했다면 남은 질문은 "싸고 믿을 만하게 돌아가는가"다.

5.1 측정한 것

표 9. 07b 실행 기록으로 본 운영 속성

속성	값	해석과 범위
LLM 비용 (GPT-6 Luna)	\$0.0714501 / 271경보 $\approx$ \$0.000264/경보 (1,000경보 $\approx$ \$0.26)	재시도 포함, 안정성 반복 제외. 평균값이며 건당 상한이 아님
JEV 비용	\$0.013059 / 271 $\approx$ \$0.000048/경보	같은 실행 기록
유효 응답	270/271 (99.6%)	같은 요청을 최대 2회 재시도한 뒤 최종 실패 1건
인용 수치 일치	성공 응답 270개의 인용 1,706개 모두 통과	필드명과 값이 입력과 맞는지만 확인. 문장의 해석이 옳은지는 보증하지 않음
반복 안정성	priority_score 표준편차 3.4~8.8점, 범위 8~19점 (100점 척도)	5개 사례 $\times$ (본 호출 1 + 반복 3)
JEV 라우팅 환산	LLM 271 $\rightarrow$ 136건 (49.8% 감소)	저장 응답으로 계산한 운영 시나리오. 실제 호출 절감 실험은 아님
라우팅 precision@E20	JEV 라우팅, 전체 LLM 모두 0.102	H2가 미검정이라 비열등성 확증이 아닌 기술 통계

5.2 역할의 정의

판단 계층에는 결정권도, 경보를 취소할 권한도 주지 않는다. LLM 응답이 실패한 경보도 대기열에서 빠지지 않는다. 산출물은 근거 요약, 반대 근거, 확인되지 않은 사항, 추가로 확인할 항목이고, 점수는 사람이 참고하는 자료다. 이번에 관측한 단가라면 rescue 경보 전부에 설명을 붙이는 방안도 검토할 만하다. 다만 사람의 검토 비용에 비해 얼마나 작은지, 통합과 감사, 유지보수에 드는 비용은 재지 않았으므로 "총비용이 무시할 만하다"고까지 말하지는 않겠다.

측정하지 않은 것. 설명을 붙였을 때 사람의 판단이 더 정확해지는지, 검토가 빨라지는지, 모델을 적절히 믿게 되는지는 재지 않았다. 이 보고서가 말하는 "비용 효율"의 근거는 낮은 API 비용과 인용 수치를 검증할 수 있다는 점까지다. 실제 효용은 사용자 연구로 확인할 과제다.

근거: artifacts/07b_rescue_lane_confirmatory/{summary.json, llm_noRF_results.jsonl, llm_stability.csv, exploratory.csv} · src/fabjudge/pipeline/claims.py · docs/jev_final_model_design.md

6. 방법론적 교훈과 평가 함정

결과가 부정적일수록 그 결과를 믿을 수 있는지가 중요하다. 이 프로젝트에서 실제로 걸려 넘어진 함정들을 정리해 둔다.

표 10. 확인한 함정과 처리

단계	함정	왜 문제인가	처리
05a → 05b	작은 표본의 낙관	45개 창에서 0.234였던 점수 차가 270개 창에서 0.037로 줄어듦	표본을 키운 결과를 기준으로 삼음
05c	같은 개발 창의 반복 재사용	10번의 시험이 선택 편향을 가짐. 남은 독립 그룹이 없어 최종 확인 불가	INCONCLUSIVE로 기록
06c	in-sample 성격의 dev로 정한 임계값	낙관적인 dev 성능이 test에서 무너짐	규칙을 바꾸지 않고 실패로 기록
06d	dev 요청에 in-sample RF 사용	J2/S2의 dev 구분력이 오염됨	결과 무효, 06e에서 정정
06a~06d	5,000초 경계 바로 밖의 이른 경보	공통 80행의 LSTM FP 8건 중 7건이 5,546~8,344초 안에 고장	공식 평가에서는 FP로 유지
07a	표본 상한 때문에 fault1/2 양성 0건	라벨로 판별력을 확증할 수 없음	prereg v2에서 표본 설계 수정
07b	고장 시각 기준으로 잘린 시퀀스의 위치 누수	시퀀스 안의 위치가 미래 고장 시각과 기계적으로 연결됨	진단용으로만 사용
06e 설계	시퀀스 시작 기준 baseline	같은 위치 누수를 가질 수 있음	가능성으로 기록

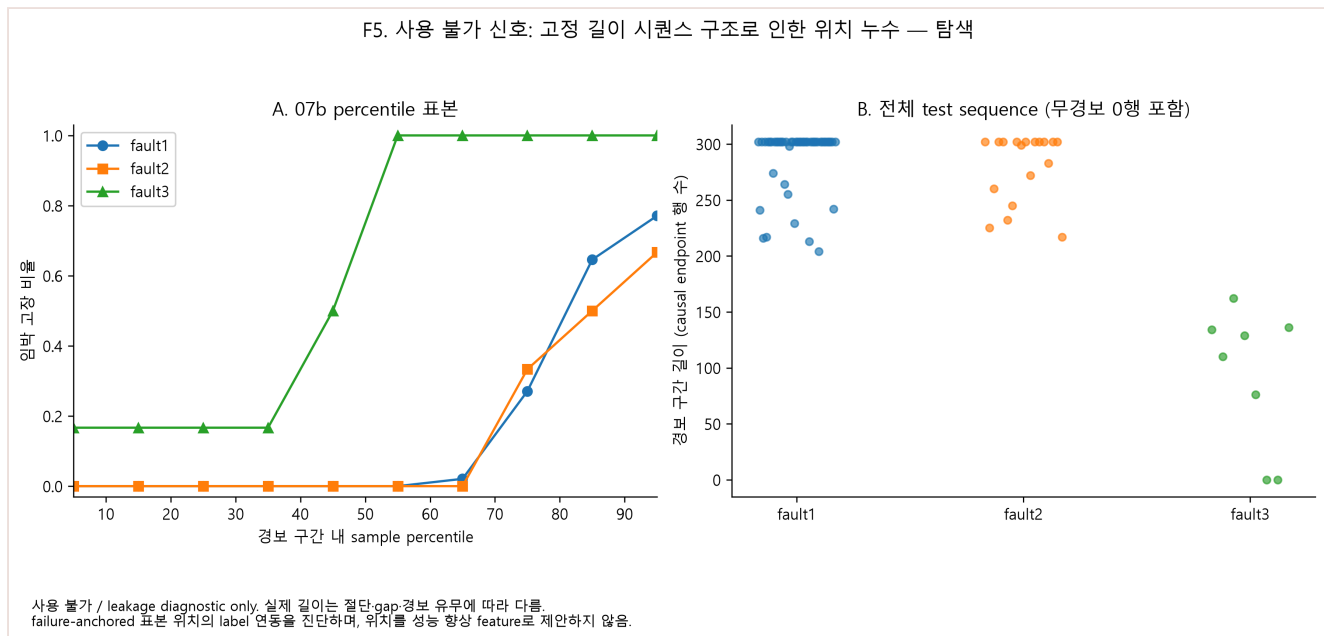


그림 F5. 쓸 수 없는 신호: 고정 길이 시퀀스가 만든 위치 누수. (A) 07b 표본에서 경보 구간 뒤쪽으로 갈수록 임박 고장 비율이 급증한다. (B) fault1/2 test 시퀀스 대부분의 경보 구간 길이가 약 300 endpoint로 거의 같다. 고장 시각에 맞춰 잘린 시퀀스에서 상대 위치는 곧 TTF 정보이므로, 실제 운영에서는 존재하지 않는 신호다.

07의 사전 등록은 두 번 고쳤다. v1 → v2에서는 표본 설계를 바꾸고 04b 비교를 추가했고, v2 → v2.1에서는 LLM 응답 1건이 JSON 파싱에 실패해 실험이 멈춘 뒤 재시도와 중단 규칙만 수정했다. 두 번 모두 07b 평가 라벨을 열기 전이었다. 다만 07a에서 표본에 양성이 없다는 것을 확인했고 fault3 test는 06 시리즈에서 이미 쓴 적이 있으므로, "어떤 라벨도 본 적 없다"고 쓰지는 않는다.

돌아보면 이 프로젝트에서 가장 믿을 만한 결론(3장)은 판단 계층을 검증하려고 평가를 엄격하게 짜는 과정에서 나왔다. 부정적 결과라도 누수 정정, 사전 등록, 미검정 상태, 개정 시점을 숨기지 않으면 다른 사람이 검증할 수 있는 결과가 된다.

근거: artifacts/08_final_comparison/tables/{T4_pitfalls.csv, F5_position_rates.csv, F5_alarm_lengths.csv} · artifacts/jev_prompt_optimization/diagnostics/summary.md · artifacts/07b_rescue_lane_confirmatory/{trial_log.md, resume_audit.json}

7. 한계

07b는 고정 percentile 지점을 뽑은 표본이고, 전체 endpoint의 rescue 곡선은 무작위 순서 기대값이지 실제 판단 계층의 결과가 아니다. fault3 test는 06 시리즈에서 이미 썼기 때문에 주 판정을 fault1+2로 두었다. train 구간의 LSTM 예측은 in-sample이며, RF를 교차적합으로 바꾼 것만으로 학습 경로 전체의 낙관이 사라지지는 않는다. 05 시리즈는 TTF 구간을 대리 라벨로 썼고, 실제 장비 상태 라벨은 없었다. 시퀀스 수와 장비 종류가 제한적이어서 endpoint가 많아도 독립적인 고장 사건 수는 적다. 5,000초라는 하나의 기준과 한정된 모델, 프롬프트, 입력 구성에서 나온 결과이며, 사람을 대상으로 한 평가는 없었다.

8. 결론과 향후 과제

세 가지로 정리한다. 첫째, 이 데이터와 설정에서는 RF 게이트를 경보를 버리는 조건이 아니라 우선순위 신호로 써야 한다. fault3 recall이 0.629에서 0.974로, fault1+2의 사건 누락이 6건에서 0건으로 바뀐다. 둘째, JEV와 LLM은 입력 표현, 스케일, 문구, 해석 규칙, 파이프라인 위치를 바꿔 가며 시험했지만 해당 데이터로 학습한 소형 모델을 대체할 근거는 나오지 않았다. 셋째, 대신 낮은 API 비용과 검증 가능한 인용을 갖춘 참고 계층으로는 둘 수 있다. 그 가치는 아직 사람을 대상으로 확인하지 않았다.

이 한계 안에서 "JEV와 LLM은 쓸모없다"는 말은 정확하지 않다. 더 정확한 결론은 **현재 실험에서 우선순위 개선을 확인하지 못했고, 설명 계층이 사람에게 주는 효용은 아직 검증되지 않았**다는 것이다.

다음 실험은 세 가지로 생각하고 있다. 첫째, 고장 시간이나 고장 기준 위치를 쓰지 않는 causal 추세 feature를 설계하고, 고장·장비·시간을 분리한 새 평가 세트를 확보한다. 둘째, 그 위에서 같은 입력을 받은 로지스틱, RF, LSTM, JEV, LLM을 나란히 비교해 모델 차이와 정보 차이를 분리한다. 셋째, 설명 제공 여부를 무작위로 배정한 사용자 실험으로 판단 정확도, 검토 시간, 과신 정도를 잰다. 경보 보존 구조가 실제로 쓸 만한지는 모델 점수 하나가 아니라 허용할 수 있는 검토량과 놓친 사건의 비용을 함께 놓고 판단해야 한다.

9. 참고문헌과 외부 자료

학술논문

1. Huang, W., Khorasgani, H., Gupta, C., Farahat, A., & Zheng, S. (2018). Remaining Useful Life Estimation for Systems with Abrupt Failures. *Annual Conference of the PHM Society*, 10(1). <https://doi.org/10.36001/phmconf.2018.v10i1.590-04b> baseline의 구조적 참고(Huang-inspired, exact reproduction 아님).
2. Singh, K., Selvanathan, B., Zope, K., Nistala, S. H., & Runkana, V. (2018). Concurrent Estimation of Remaining Useful Life for Multiple Faults in an Ion Etch Mill: A Data-driven Approach. *Annual Conference of the PHM Society*, 10(1). <https://doi.org/10.36001/phmconf.2018.v10i1.591-related-work>.
3. T. V., V., Gupta, P., Malhotra, P., Vig, L., & Shroff, G. (2018). Recurrent Neural Networks for Online Remaining Useful Life Estimation in Ion Mill Etching System. *Annual Conference of the PHM Society*, 10(1). <https://doi.org/10.36001/phmconf.2018.v10i1.589-RNN-related-work>.

데이터셋과 공식 문제 정의

4. PHM Society. (2018). PHM Data Challenge 2018 — Ion Mill Etch Tool. <https://phmsociety.org/conference/annual-conference-of-the-phm-society/annual-conference-of-the-prognostics-and-health-management-society-2018-b/phm-data-challenge-6/>

기술문서 (동료 심사 아님)

5. OpenRouter / TypeSafe. What Is Jev? <https://openrouter.ai/blog/insights/what-is-jev/>
6. OpenRouter. How to Use Jev. <https://openrouter.ai/blog/tutorials/how-to-use-jev/> — 실행 기록상 모델: JEV typesafe/jev-1.13 (반환 typesafe/jev-1.13-20260917), LLM GPT-6 Luna (API 모델 ID ~openai/gpt-luna-latest).

공개 구현 참고

7. ninja1mmm. 2018-phm-data-challenge [GitHub repository]. <https://github.com/ninja1mmm/2018-phm-data-challenge>
— 공식 우승 코드나 특정 논문의 공식 구현으로 표현하지 않는다.

이 프로젝트에서 설계한 부분

- JEV-LLM을 기존 탐지·RUL 모델의 대체가 아닌 선택적 판단 계층으로 검증한 연구 질문
- JEV 단독 실험에서 가설 프롬프트를 고정 검증기와 사전 기준으로 시험한 절차(05c)
- RF 게이트가 버린 LSTM 경보를 rescue 모집단으로 정의해 tier1과 분리 평가한 구조
- noRF EvidencePacket, H1→H2→H3 순차 검정, 정수 검토 예산, 순위 해시 고정, 라벨 개봉 감사를 포함한 07b 확증 프로토콜
- endpoint 단위와 사건 단위 탐지를 분리하고, 전체 endpoint 사후 비교와 확증 표본을 구분한 평가 체계
- LLM이 인용한 수치를 원 입력과 자동 대조하는 검증 절차

이 항목들이 새롭다고 주장하려면 PHM·LLM 의사결정 지원 분야의 문헌 조사가 먼저 필요하다.

부록

A. 실험 연표

본문과 부록의 경로는 [GitHub 저장소](#) 루트 기준이다. 노트북, 소스 코드, 07b·08 결과 표와 그림은 저장소에 있고, 원시 측정값이나 API 요청·응답 원문이 담긴 일부 artifact는 공개 범위에서 제외했다.

단계	실행 파일	핵심 기록
05a	notebooks/05a_jev_tut.ipynb	artifacts/jev_tut/config.json, representation_summary.csv
05a 확장	src/fabjudge/jev_scale_extremes.py	artifacts/jev_scale_sensitivity/extreme_0p1_10x/summary.md
05b	notebooks/05b_jev_prompt_optimization.ipynb	artifacts/jev_prompt_optimization/run_summary.md
05c	notebooks/05c_jev_v4_edge_trials.ipynb	artifacts/jev_v4_edge_trials/trial_log.md
06a	notebooks/06a_lstm_jev_hybrid.ipynb	artifacts/06a_lstm_jev/larger_test_01/summary.json
06b	notebooks/06b_lstm_jev_sensor_skip.ipynb	artifacts/06b_lstm_jev_sensor_skip/summary.json
06c	notebooks/06c_lstm_jev_semantic_gate.ipynb	artifacts/06c_lstm_jev_semantic_gate/summary.json
06d	notebooks/06d_lstm_jev_oof_edge.ipynb	artifacts/06d_lstm_jev_oof_edge/ERRATUM.md
06e	notebooks/06e_fix_and_weakness_diagnosis.ipynb	artifacts/06e_fix_and_weakness_diagnosis/conclusion.md
07a	scripts/run_07a.py	artifacts/07a_pipeline_skeleton/conclusion.md
07b	notebooks/07b_rescue_lane_confirmatory.ipynb	artifacts/07b_rescue_lane_confirmatory/conclusion.md
08	notebooks/08_final_comparison.ipynb	artifacts/08_final_comparison/summary_ko.txt

B. 사전 등록 해시

파일	SHA-256
jev_v4_edge_trials/validation_spec.json (v1.2)	1e0920672741dc86f97f0dc241c8d286b9ab6067ec976c993b8a9b7a6865600e
prereg_07.json (v1)	9e65c4a8220248b01108e2124697ad282bc6e17c58cfd7072d952484a127e725
prereg_07_v2.json	9eb9fe21d46451d1bc022c75d28436460d6b411b425e005a6bd8e14d31524cbe
prereg_07_v2_1.json	5112fdb2be2a1618cc52ad3c5e8932d974a2362f04fa245204c5562f406942eb
rankings.csv (순위 고정)	26ffc1a4d6bea99fde201a26c784927dd6529e9c576479930478af8eb6fa36eb

C. API 호출과 비용

실행	호출	기록 비용 (USD)
05a 입력 표현	141	0.0059
05a 스케일 (0.5×/2×)	140	0.0058
05a 스케일 (0.1×/10×/신규 1×)	135	0.0056
05b 프롬프트 (v4/v5)	552	0.0235
05c 가설 프롬프트 10종	450	0.0210
07b (JEV 271, LLM 282, 안정성 15)	568	0.0881

06 시리즈와 07a, 05b의 Noul 후속 실행은 각 artifact의 기록을 따로 참조한다. 위 표는 프로젝트 전체의 누적 비용이 아니다. 07b의 cost_curve는 "검토 건수 + FN × 누락 가중치"를 비교한 것으로 달러 비용 표가 아니다.

D. 04b 원래 지표와 재현 범위

파일 이름과 달리 04b는 exact reproduction으로 표기하지 않는다. 04b의 원래 지표(T5b: RF의 5k fault 대 50k 초과 normal 분류, IPW 지표, sample-count RUL 회귀, combined 지표)는 본문의 전체 endpoint TTF 평가와 정의가 달라 직접 비교하지 않았다. 04b의 R²는 저장된 값이 없어 다른 값으로 대체하지 않았다.

E. 최종 07b EvidencePacket (16개 필드)과 출력 스키마

구분	필드
LSTM 요약 (초 단위)	lstm_pred_seconds, recent_lstm_delta, recent_lstm_std
현재 센서 (정규화 입력)	ROTATIONSPEED, IONGAUGEPRESSURE, ETCHSUPPRESSORCURRENT, FLOWCOOLPRESSURE, ETCHBEAMCURRENT, FLOWCOOLFLOWRATE
과거 100행 요약	ROTATIONSPEED__rms, IONGAUGEPRESSURE__shape_factor, ETCHSUPPRESSORCURRENT__peak_abs, FLOWCOOLPRESSURE__shape_factor, FLOWCOOLPRESSURE__peak_abs, FLOWCOOLFLOWRATE__peak_abs
fault family	fault_type

출력 필드는 evidence_for, evidence_against, unknowns, additional_checks, priority_score(0~100), review_recommendation(urgent_review / standard_review / low_priority_review)이다. 근거 항목에는 field, value, claim이 들어간다. 모두 검토 권고일 뿐 고장 확정이나 경보 취소 명령이 아니다. 07a에 있던 rf_probability는 07b에서 뺐다.

원본: src/fabjudge/pipeline/schemas.py · artifacts/07a_pipeline_skeleton/{packet_schema.json, llm_output_schema.json} · artifacts/07b_re
scue_lane_confirmatory/{prereg_07_v2_1.json, actual_request_body_example.json, questions/R2.json}